Disjoint sets

Part V Advanced Data Structures

Scope

Disjoint sets turn a partition that only merges into representative queries. We compare weighted lists and rank forests on the same merges, then prove the inverse-Ackermann sequence bound through complete level/iter potential accounting. Single-operation cost, amortized sequence cost, and implementation assumptions remain distinct.

Prerequisites

Representatives, forests, ranks, and path compression.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

Learning objectives

- Compare linked-list and forest representations.
- Apply union by rank and path compression.
- Prove the inverse-Ackermann amortized bound.
- 21.1 Disjoint-set operations
- 21.2 Linked-list representation of disjoint sets
- 21.3 Disjoint-set forests
- 21.4 Analysis of union by rank with path compression

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The partition is the state; a representative is a handle

Every established universe element belongs to exactly one nonempty set, and these sets are pairwise disjoint. MAKE-SET(x) requires a new element and creates {x}. FIND-SET(x) requires an existing element and returns a representative of its set. UNION(x,y) merges two sets. The representative need not be the minimum key, and internal parent pointers need not be original graph edges.

Let n count MAKE-SET calls and m count all external MAKE-SET, FIND, and UNION calls, with $m \geq n$. Establish all n singletons first to fix the analysis parameters. There are at most $n-1$ successful merges, but arbitrarily many repeated UNION calls. Find both roots first; if they agree, return without LINK or a rank increase. LINK accepts **two distinct roots**.

FIND can compress parents without changing the partition. UNION only merges. Deletion, rollback, dynamic removal of graph edges, and enumeration of all members require additional structures or operations. Different sets have different representatives; all finds within one set return the same representative.

Why union maintains exactly the connected components

For an undirected graph, MAKE-SET every vertex and UNION(u,v) for each arriving edge. The invariant is that the DSU partition equals the connected components of the processed-edge subgraph. Initially both consist of singletons. An edge within a component changes neither structure; an edge between components joins exactly those two. Induction proves that at termination FIND(u)=FIND(v) exactly when an undirected u–v path exists.

On a..f, insert ab, cd, bc, ab, ef, de. The partitions are {ab},{c},{d},{e},{f}; {ab},{cd},{e},{f}; {abcd},{e},{f}; the same after repeated ab; {abcd},{ef}; and {abcdef}. Five merges succeed during six UNION calls.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why union maintains exactly the connected components (continued)

Edge stream: each partition change

Each row shows the partition after an edge. Repeated edge 4 leaves the set count fixed.

			Set count
ab	⇒	{a,b} {c} {d} {e} {f}	5
cd	⇒	{a,b} {c,d} {e} {f}	4
bc	⇒	{a,b,c,d} {e} {f}	3
ab	⇒	{a,b,c,d} {e} {f}	3
ef	⇒	{a,b,c,d} {e,f}	2
de	⇒	{a,b,c,d,e,f}	1

Five merges: 6→5→4→3→3→2→1. All six UNION calls still check representatives.

Static DFS or BFS already finds components in $\Theta(n+|E|)$. DSU additionally supports connectivity queries as edges arrive and Kruskal cycle tests. Ignoring directions in a directed graph gives weak connectivity, not strongly connected components.

21.2 Linked-list representation of disjoint sets

Set records and member back-pointers

Each set record stores head, tail, and size; each member stores next and a set pointer. The representative is record.head. An empty collection takes O(1) to initialize. MAKE-SET creates next=NIL, head=tail=x, size=1, and x.set pointing to the new record. FIND returns x.set.head in O(1).

UNION reads S=x.set and T=y.set. If equal, return the head. Swap S,T if S.size<T.size; on ties keep the first S. Set S.tail.next=T.head, S.tail=T.tail, and S.size+=T.size. Traverse the old T list, redirecting each member's set pointer to S, then discard T's record. The traversal retains the original next links, so successors are not lost. Tail splicing alone does not update representatives.

One UNION costs Θ(1+min(|S|,|T|)). Merging a→b→c with d→e produces a→b→c→d→e. Only d,e change set records; the tail moves from c to e and size from 3 to 5. Their FIND calls now return a.

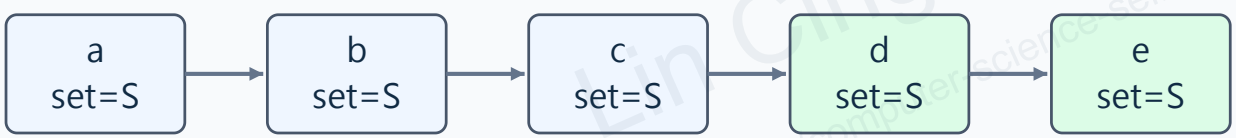
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Weighted lists: update set as well as next

Before: S head=a, tail=c, size 3; T head=d, tail=e, size 2.



Append short to long. Change only d,e set pointers to S, and set c.next=d.



After: S head=a, tail=e, size 5. Green members changed set; arrows show next.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A quadratic counterexample and the doubling proof

Suppose the second set is always appended to the first. Starting with n singletons, UNION(x_2,x_1), UNION(x_3,x_2), … makes each new singleton absorb the old long list. Pointer rewrites number 1,2,…,n−1, totaling n(n−1)/2. Thus m=2n−1 calls cost $\Theta(n^2)$, despite constant-time next splicing with a tail pointer.

Weighted union moves only the shorter set. Each rewrite of a fixed member x at least doubles its set size. After k moves, size is at least 2^k and at most n, so $k \leq \text{floor}(\log_2 n)$. Summing over n members gives at most n floor($\log_2 n$) rewrites, plus O(m) call overhead, for O(m+n log n). Use log(n+1) for small n. A single merge of two n/2-member lists still costs Θ(n); the sequence bound does not give worst-case O(log n) per union.

For $n=2^k$, repeatedly pair equal-sized sets. Each of k rounds rewrites exactly n/2 pointers, totaling $(n/2)\log_2 n$. Constant-time MAKE and FIND are consistent with this total union cost.

Accounting for eight elements

Keep the first set on equal sizes. Use this sequence again for the forest. Letters are handles, not priority keys.

UNION	Old sizes	Members whose set changes	New representative/size	Cumulative rewrites
a,b	1,1	b	a／2	1
c,d	1,1	d	c／2	2
e,f	1,1	f	e／2	3
g,h	1,1	h	g／2	4
a,c	2,2	c,d	a／4	6
e,g	2,2	g,h	e／4	8
a,e	4,4	e,f,g,h	a／8	12

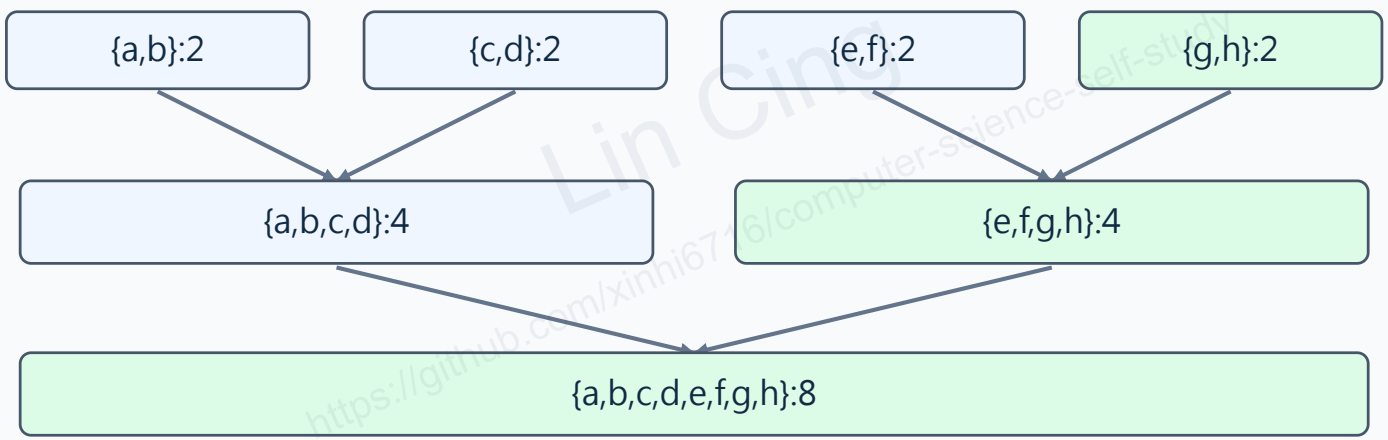
Lin Cing https://github.com/xinhi6716/computer-science-self-study

Accounting for eight elements (continued)

For h, the set pointer follows h→g→e→a, and the containing size grows 2→4→8. It changes three times; a never changes. Total rewrites are 12, not three for every element. In the earlier singleton-absorption sequence, weighting reduces the total to n−1 rewrites.

Equal-size pairwise merges: three rounds of rewrites

Each round rewrites four set pointers. Green marks h's containing set: 2→4→8.



Total rewrites 4+4+4=12; h changes 3 times, a changes 0 times. Sets never split.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Self-parent roots and the LINK precondition

Each element stores parent and rank; a root points to itself. MAKE-SET sets parent[x]=x and rank[x]=0. UNION must first find both roots. Linking arbitrary x directly to y can split an existing set or create a cycle.

```
UNION(x,y):
    r=FIND(x); s=FIND(y)
    if r==s: return r
    return LINK(r,s)
LINK(r,s):                    // r and s are distinct roots
    if rank[r]<rank[s]: swap r,s
    parent[s]=r
    if rank[r]==rank[s]: rank[r]+=1
    return r
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

21.3 Disjoint-set forests (continued)

After linking roots, all old members still reach the new root. One root loses its self-parent, with no reverse path creating a cycle. Unequal ranks leave the retained rank unchanged; equal ranks increase it by one. Rank forests and size-weighted lists are different representations. Rank is not exact set size.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Two-pass FIND preserves the old parent before overwriting

```
FIND(x):
    r=x
    while parent[r]!=r: r=parent[r]
    while parent[x]!=x:
        old=parent[x]
        parent[x]=r
        x=old
    return r
```

The first pass finds the root without mutation. The second follows the old route and redirects every parent to r. Overwriting parent[x] before saving it jumps immediately to r and misses intermediate nodes. Recursive parent[x]=FIND(parent[x]) finds the root upward and redirects while returning. Its stack uses at most the path height; the two-pass iterative version uses O(1) extra space.

Compression reconnects nodes only to the root of the same tree, preserving roots and the partition. It never changes rank. Visiting s nodes costs Θ(s). Forest storage is O(n). Member lists can support enumeration, but parent pointers alone do not automatically provide O(set size) enumeration.

The same eight elements before and after compression

For the seven earlier unions, equal ranks retain the first root. The successive root ranks are a1,c1,e1,g1,a2,e2,a3. Final parents are b→a, c→a, d→c, e→a, f→e, g→e, h→g; a points to itself.

Vertex	a	b	c	d	e	f	g	h
rank	3	0	1	0	2	0	1	0
Parent before FIND(h)	a	a	a	c	a	e	e	g
Parent after FIND(h)	a	a	a	c	a	e	a	a

The first pass takes h→g→e→a, three links. The second sets the parents of h,g,e to a; e already had that value. Nodes d→c and f→e are outside this path and do not change. All ranks remain fixed. A later FIND(h) takes one link, but FIND(d) still takes two. One find does not flatten the whole tree.

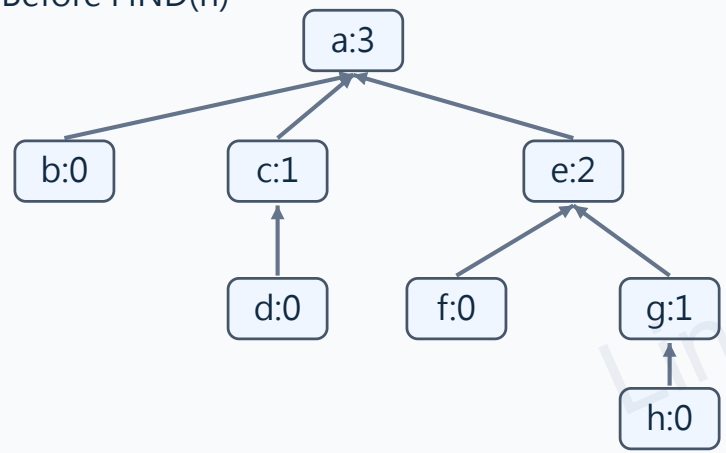
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The same eight elements before and after compression (continued)

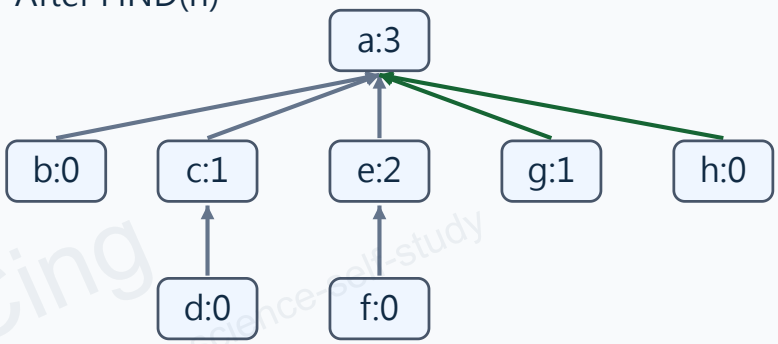
The same eight-node tree: parents before and after compression

Nodes show key:rank; arrows show parent; root self-loops omitted; green links changed.

Before FIND(h)



After FIND(h)



h→g→e→a; only h,g links change. Node e already points to a; d,f are off-path.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Rank bounds height and historical membership

Induct on operations to prove that a root's current height is at most its rank. A singleton has height zero. For unequal ranks, the smaller height plus one is at most the larger rank; equal ranks allow one extra level. Compression only shortens paths. A rank-r root contains at least 2^r members: its rank rises only by merging two rank-(r−1) sets, each of size at least 2^{r-1}. Other merges only add members. A nonroot keeps its rank and the historical membership bound from when it was last a root. Thus every rank is at most $\text{floor}(\log_2 n)$.

Rank is not a lower bound on current height. After compressing every element in the example, a retains rank 3 but has height 1. A rank-2 root can likewise be the center of a four-node star. Rank is not current subtree size either, since compression can move descendants away without reducing rank.

Rank alone gives O(log n) per FIND and O(m log n) overall. Pairwise equal-rank merges create height $\log_2 n$, and repeated uncompressed finds on a deepest leaf attain that order. Without rank, arbitrary links can create a linear chain and a first FIND of Θ(n). That counterexample does not apply when both heuristics are present.

Ackermann iteration means composition

Fix the final number of elements n and set $\alpha = \alpha(n)$ throughout the analysis, even while MAKE-SET calls are still creating elements. For $j \geq 1$, define

$$f^{(0)}(j) = j, \quad f^{(i+1)}(j) = f(f^{(i)}(j)), \quad A_0(j) = j + 1, \quad A_k(j) = A_{k-1}^{(j+1)}(j) \ (k \geq 1).$$

The superscript counts **repeated applications to the previous output**. Since $A_0^{(i)}(j) = j + i$, we get $A_1(j) = 2j + 1$. Induction gives $A_1^{(i)}(j) = 2^i(j + 1) - 1$, hence

$$A_2(j) = 2^{j+1}(j + 1) - 1, \qquad A_3(1) = A_2(A_2(1)) = A_2(7) = 2047.$$

Inductively, each A_k is strictly increasing in its input and satisfies $A_k(j) \geq j + 1$: a larger starting input undergoes more iterations, each increasing its input. Since $j + 1 \geq 2$, also $A_k(j) > A_{k-1}(j)$. Both monotonicities are used below.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

The chosen inverse-Ackermann convention

Use the single-parameter convention of CLRS third edition, Section 21.4:

$$\alpha(n) = \min\{k \geq 0 : A_k(1) \geq n\}.$$

k	0	1	2	3	4
$A_k(1)$	2	3	7	2047	greater than 10^{80}

Thus $\alpha(2) = 0$, $\alpha(8) = 3$, and $\alpha(2048) = 4$. Other texts may use a two-parameter convention; its numerical values cannot be substituted here.

The complete bound is $O(m(1 + \alpha(n)))$ for m external operations, including n MAKE-SET calls. Only for $n \geq 3$ can it be shortened to $O(m\alpha(n))$. Small instances still require constant work per operation.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Which quantities can change?

Induction on operations shows that a root points to itself and every nonroot has smaller rank than its parent. Linking equal-rank roots raises the retained root's rank by one. Unequal ranks attach the smaller rank under the larger. Compression changes parents, never ranks.

Once a node becomes a nonroot, its rank is fixed. Its parent's rank may increase, or compression may replace the parent by a higher ancestor. Neither event decreases parent rank. This is why a node's level cannot decrease over time.

A rank-r root contains at least 2^r elements. A nonroot retains that historical bound from losing root status. Hence every rank is at most $\lfloor \log_2 n \rfloor < n$, regardless of the current compressed subtree size.

Define level and iter only for positive-rank nonroots. Rank-zero nodes and roots receive separate potential formulas.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Level selects the scale; iter measures progress within it

For a nonroot, write $r = \operatorname{rank}(x) \geq 1$, $p = \operatorname{rank}(\operatorname{parent}(x))$, and define

$$\ell(x) = \max\{k \geq 0 : p \geq A_k(r)\}, \qquad i(x) = \max\{j \geq 1 : p \geq A^{(j)}_{\ell(x)}(r)\}.$$

Here $\ell = \operatorname{level}$ and $i = \operatorname{iter}$. First find the highest scale reached by parent rank; then count iterations at that scale.

Since $p \geq r + 1 = A_0(r)$, we have $\ell \geq 0$. Also $A_\alpha(r) \geq A_\alpha(1) \geq n > p$, so $\ell < \alpha$. The definition gives $i \geq 1$. If $i \geq r + 1$, then

$$p \geq A^{(r+1)}_\ell(r) = A_{\ell+1}(r),$$

contradicting maximality of ℓ. Therefore $0 \leq \ell < \alpha$ **and** $1 \leq i \leq r$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Nonnegative potential records work still available

Assign a potential to each node and sum over the forest:

$$\phi(x) = \begin{cases} \alpha r, & x \text{ is a root or } r = 0, \\ (\alpha - \ell(x))r - i(x), & x \text{ is a nonroot and } r \geq 1, \end{cases} \qquad \Phi = \sum_x \phi(x).$$

For nonroots, the lower bound is $(\alpha - (\alpha - 1))r - r = 0$, and the upper bound is $\alpha r - 1$. Roots have potential αr; rank-zero nodes have zero potential. Thus Φ is always nonnegative.

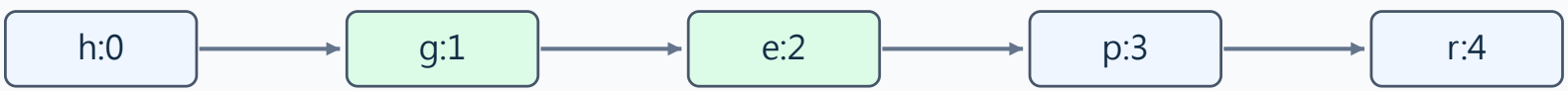
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Potential accounting on a five-node path

Take $n = 16$, $\alpha = 3$, and compress a path of ranks 0, 1, 2, 3, 4. The root keeps its self-parent.

FIND values: progress through level and iter

n=16, α=3; after FIND(h), h,g,e,p all point to r. All ranks stay fixed.



Before (level,iter)				
—	(0,1)	(0,1)	(0,1)	root
After (level,iter)				
—	(1,1)	(0,2)	(0,1)	root
φ 0→0	φ 2→1	φ 5→4	φ 8→8	φ 12→12

Good nodes g,e: path potential 27→25. Exceptions: rank 0, root, last node at each level.

Each good node releases one unit: g gains a level; e increases iter within one level. The other three potentials stay fixed, so the path total falls from 27 to 25.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Why can a rising level reset iter?

Fix the rank r of a nonroot. Within one level, increasing parent rank leaves iter unchanged or increases it. Each additional iteration decreases ϕ by one.

If the level rises from k to $k' > k$, iter may reset from i to a smaller i'. Nevertheless,

$$\Delta\phi = -(k' - k)r + (i - i') \leq -r + (r - 1) = -1.$$

This uses $k' - k \geq 1$, $i \leq r$, and $i' \geq 1$. **The gain from changing level pays for resetting iter.** The pair progresses in lexicographic order, comparing level first, then iter. Its two coordinates need not increase independently.

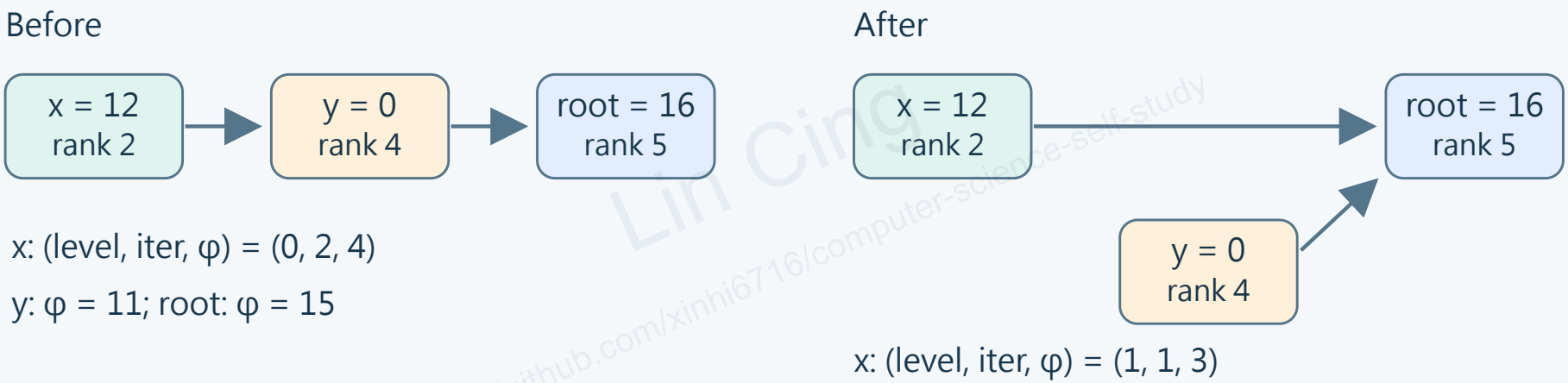
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A legal FIND that really decreases iter

Take $n = 32$ and $\alpha = 3$. The find path is $12 \to 0 \to 16$, with ranks $2, 4, 5$. Unvisited branches are omitted and remain unchanged.

A higher level can reset iter from 2 to 1

n=32, α=3; FIND(12)



Only the find path is drawn; omitted branches are unchanged.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Computing the reset example term by term

Node $x = 12$ keeps rank 2. Only its parent rank changes, from 4 to 5.

Quantity	Before compression	After compression
level	$A_0(2) = 3 \le 4 < A_1(2) = 5$, so 0	$A_1(2) = 5 \le 5 < A_2(2) = 23$, so 1
iter	$A_0^{(2)}(2) = 4 \le 4 < A_0^{(3)}(2) = 5$, so 2	$A_1(2) = 5 \le 5 < A_1^{(2)}(2) = 11$, so 1
$\phi(x)$	$(3 - 0)2 - 2 = 4$	$(3 - 1)2 - 1 = 3$

Node 0 keeps potential 11, and the root keeps potential 15. All other branches are unchanged. Thus the forest has $\Delta\Phi = (3 + 11 + 15) - (4 + 11 + 15) = -1$. The displayed totals sum only the path; the other 29 nodes need not have zero potential.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Constructing that path with legal operations

Create singleton keys 0 through 31. Equal-rank merges retain the first root.

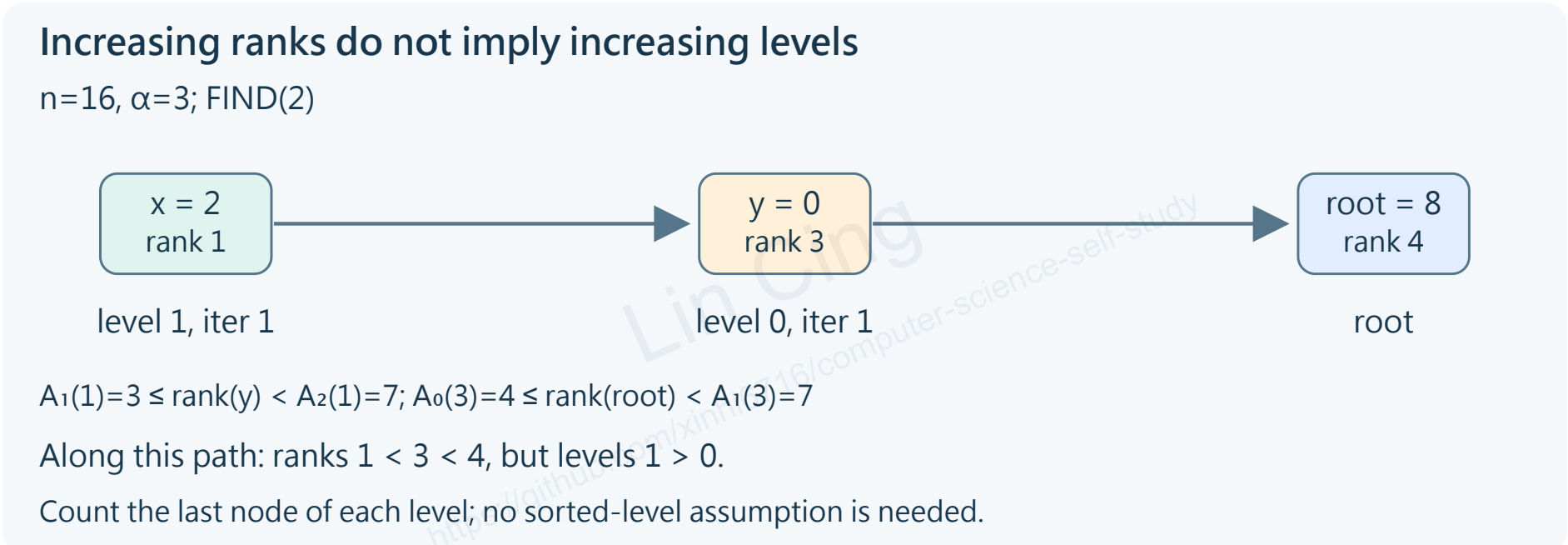
Operation	Checkable result
Merge adjacent equal-size blocks of 0..15 in rounds	Root 0 has rank 4; node 12 has rank 2
FIND(12)	Parent of 12 becomes 0; ranks stay fixed
Build 16..31 by the same rounds	Root 16 has rank 4
UNION(16,0)	Root 16 rises to rank 5; parent of 0 becomes 16
FIND(12) again	Path $12 \to 0 \to 16$; parent of 12 becomes 16

The first half starts with (0,1), (2,3), …, (14,15), then (0,2), (4,6), …, followed by (0,4), (8,12), and finally (0,8). These ranks come from a legal history rather than arbitrary assignments to a drawn chain.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Levels need not increase along a path

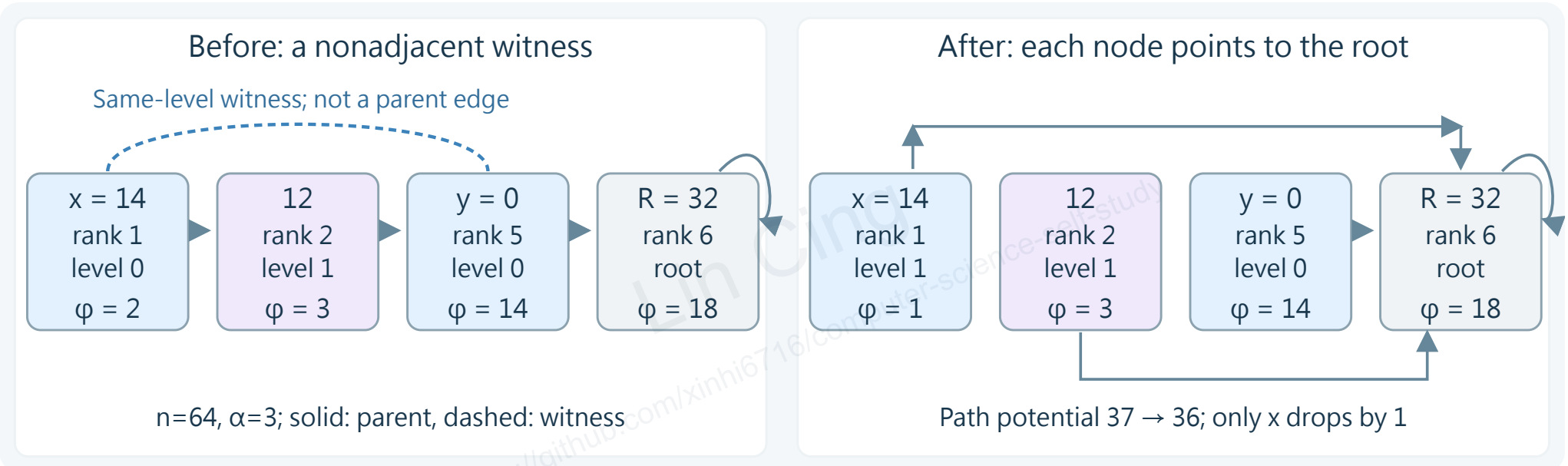
Take $n = 16$. Merge 0..7 into root 0 of rank 3, then call FIND(2). Build 8..15 into root 8 of rank 3 and call UNION(8,0). This creates the following legal path.



Comparing different nodes changes the input rank too. Temporal monotonicity concerns one fixed node.

A same-level witness need not be the next node

A positive-rank nonroot x is good when a later nonroot y on the old path satisfies $\ell(x) = \ell(y)$. **Look for a later node of the same level, not just the next node.**



Path $14 \to 12 \to 0 \to 32$ has ranks 1,2,5,6 and levels 0,1,0; the root has no level. Node 0 witnesses that 14 is good even though node 12 has another level. Compression can give 14 a higher parent rank without making these two nodes adjacent beforehand.

Construct the path through legal operations

Start with 64 singletons. Equal-rank UNION retains its first root. Building an interval in rounds means merging adjacent singletons, then adjacent equal-sized blocks.

Operation	Fact established by the history
Build 0..31 in rounds	Root 0 has rank 5; node 12 has rank 2 and node 14 rank 1
FIND(12)	Old path is $12 \to 8 \to 0$; attach 12 to 0, while 14 still points to 12
Build 32..63 in rounds	Root 32 has rank 5
UNION(32,0)	Equal ranks: 32 rises to 6, and 0 points to 32
Before FIND(14)	Path $14 \to 12 \to 0 \to 32$, with ranks 1,2,5,6

FIND(12) follows $12 \to 8 \to 0$ upward to the root, then attaches each visited nonroot directly to that root. It never enters a child subtree, so 14 still points to 12. The later union attaches root 0 to root 32. These ranks follow from legal operations; 32 retains its self-parent.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Compute every level, iteration count, and potential

Fix $n = 64, \alpha = 3$. Level and iter apply only to positive-rank nonroots; the root uses $\phi = \alpha r$.

Node	Rank	Parent rank: old→new	Level: old→new	Iter: old→new	ϕ: old→new
14	1	2→6	0→1	1→1	2→1
12	2	5→6	1→1	1→1	3→3
0	5	6→6	0→0	1→1	14→14
32, root	6	6→6	Not defined	Not defined	18→18

For 14, initially $A_0(1) = 2 \le 2 < A_1(1) = 3$, so level=0,iter=1. Afterwards $A_1(1) = 3 \le 6 < A_2(1) = 7$, and $A_1^{(2)}(1) = 7 > 6$, so level=1,iter=1. Its potential drops from $(3-0)1 - 1 = 2$ to $(3-1)1 - 1 = 1$.

For 12, $5 = A_1(2) \le p < 11 = A_1^{(2)}(2) < A_2(2) = 23$, so level=1,iter=1. For 0, $A_0(5) = 6 \le 6 < A_0^{(2)}(5) = 7 < A_1(5) = 11$, so level=0,iter=1,potential=14. Path total changes $37 \to 36$. All other nodes keep rank,parent rank,root status; **the whole forest also loses exactly one unit**.

The good-node inequality, with a reason for each step

Freeze all quantities before compression. Write $\rho(v) = \mathrm{rank}(v)$, $p_x = \mathrm{parent}(x)$, $p_y = \mathrm{parent}(y)$, and let R be the root. Put $r = \rho(x), k = \ell(x) = \ell(y), i = i(x)$.

Step	Reason
$\rho(R) \geq \rho(p_y)$	p_y is on the root path; rank strictly increases along nonroot parent edges
$\rho(p_y) \geq A_k(\rho(y))$	Definition of $\ell(y) = k$
$A_k(\rho(y)) \geq A_k(\rho(p_x))$	y equals or is above p_x, so $\rho(y) \geq \rho(p_x)$; A_k increases
$A_k(\rho(p_x)) \geq A_k(A_k^{(i)}(r))$	Iter gives $\rho(p_x) \geq A_k^{(i)}(r)$; use monotonicity again
$A_k(A_k^{(i)}(r)) = A_k^{(i+1)}(r)$	One more function composition, not multiplication

The new parent rank is therefore at least $A_k^{(i+1)}(r)$. No step assumes sorted path levels: the proof uses ancestor rank order and monotonicity at one fixed level k.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Crossing a threshold pays for a good node

Compression makes R the parent of x, keeping r fixed. The proof crosses the **next iteration threshold of the old level**. It does not guarantee that the new iter exceeds the old one; distinguish two cases.

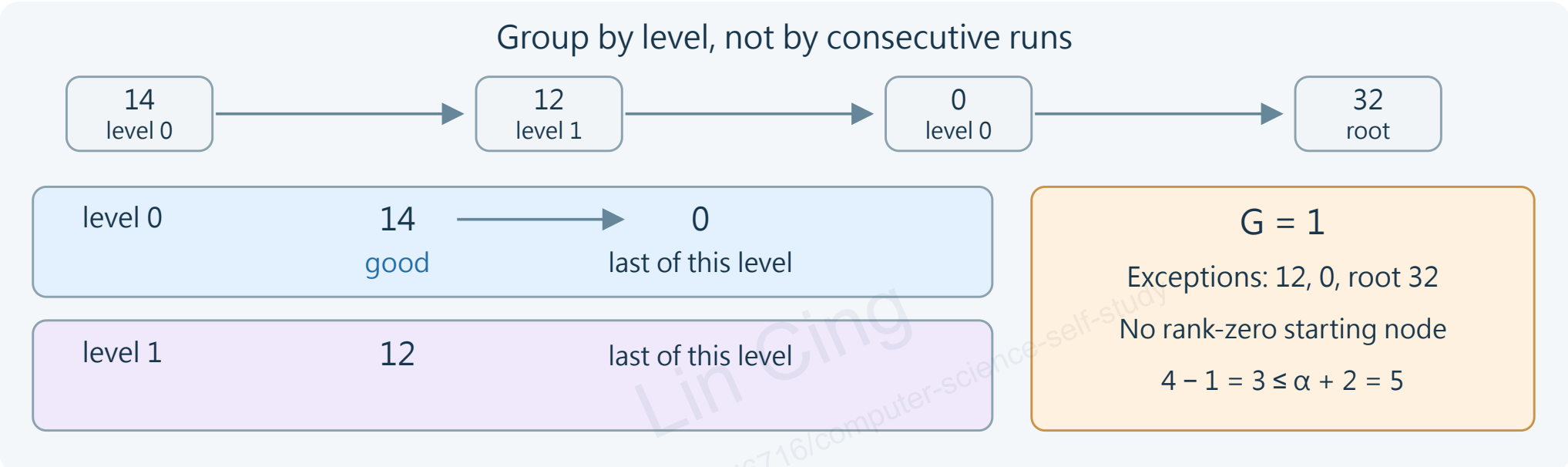
After compression	Potential change	Why it decreases
Level stays k	$\Delta\phi = -(i' - i) \le -1$	Iter is at least $i + 1$
Level rises	$\Delta\phi = -(k' - k)r + i - i'$	$k' - k \ge 1, i \le r, i' \ge 1$, so $\Delta\phi \le -r + (r - 1) = -1$

In the 64-node example, $x = 14, y = 0, k = 0, i = 1$; the inequalities become $6 \ge 6 \ge 6 \ge 3 \ge 3$. The next old-level threshold is $A_0^{(2)}(1) = 3$. New parent rank 6 also crosses $A_1(1) = 3$, so this is the second case and potential drops by one.

Charge the decrease to good node x, not again to witness y; y's potential need not drop. Other nodes' potentials do not increase, so the good-node decreases may be summed.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Only the last node of each level is an exception



Within one level, every node except the last has a later same-level witness. Here 14 is good; 12 and 0 are the last nodes of levels 1 and 0. Adding root 32 gives three exceptions. Treating consecutive runs 0/1/0 as three groups would miss good node 14.

Convert group counts into total potential decrease

An s-node path has at most α positive-rank nonroot levels, numbered $0, \dots, \alpha - 1$. At most one last node per level, the root, and a possible rank-zero starting node give at most $\alpha + 2$ exceptions. Strictly increasing ranks allow at most one rank-zero node.

$$G \geq g := \max(0, s - (\alpha + 2)), \qquad \Delta\Phi \leq -G \leq -g.$$

G counts actual good nodes; g is only a conservative lower bound. Here $s = 4, \alpha = 3$, hence $g = 0$, yet $G = 1$ and $\Delta\Phi = -1$. A short path need not attain the bound. The next argument uses the general lower bound to pay for long paths.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Fixed constants pay for the entire scan

Suppose two-pass find costs at most cs. Fix $C > c$ and use scaled potential $\Psi = C\Phi$. The amortized cost satisfies

$$\hat{c} = c_{\text{actual}} + \Delta\Psi \le cs - Cg.$$

Path length	Bound after substituting g
$s \le \alpha + 2$	$cs \le c(\alpha + 2)$
$s > \alpha + 2$	$(c - C)s + C(\alpha + 2) \le C(\alpha + 2)$

Thus each FIND has amortized cost $O(1 + \alpha)$. For $s = 7$, $\alpha = 3$, $c = 2$, $C = 3$, the lower bound is $g = 2$, so $\hat{c} \le 14 - 6 = 8$. Keep the same C throughout the sequence.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Only the retained LINK root can gain potential

MAKE-SET adds a rank-zero node with zero potential and still has amortized cost $O(1)$. Suppose LINK attaches old root u under retained root v.

Possibly affected nodes	Reason for the potential change
u	Loses root status without changing rank; potential does not rise
Old children of v	Parent rank cannot fall; the earlier monotonicity argument applies
v	Unequal ranks leave it fixed; equal ranks add one, increasing potential by α

Every other node keeps its rank, parent rank and root status. Therefore $\Delta\Psi \leq C\alpha$, and LINK costs $O(1) + C\alpha = O(1 + \alpha)$ amortized. UNION performs two FIND calls and links only distinct roots, expanding into at most three operations.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Recovering actual sequence cost

Let c_j be actual cost and $a_j = c_j + \Psi_j - \Psi_{j-1}$ amortized cost. Summing cancels intermediate potentials:

$$\sum_{j=1}^{m'} c_j = \sum_{j=1}^{m'} a_j + \Psi_0 - \Psi_{m'} \leq \sum_{j=1}^{m'} a_j.$$

The inequality uses the empty initial forest, $\Psi_0 = 0$, and nonnegative final potential. Expanding UNION gives $m' \leq 3m$, so the original m operations cost $O(m(1 + \alpha(n)))$ in total.

This is a worst-case bound on the entire sequence. With rank and compression, a single FIND can still cost $\Theta(\log n)$: build a balanced rank tree and first query a deepest leaf. A linear chain requires removing the rank rule and does not satisfy this theorem's hypotheses.

Potential analysis reference: CLRS, third edition, Section 21.4, printed pp. 576–581.

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study